Module 5

CSS Box Model and Layout

Agenda

1. CSS Box Model
2. Margin, Padding and Borders
3. Width, Height and Box Sizing
4. Display Properties
5. CSS Positioning
6. Overflow and Z-Index

© Copyright Intellipaat. All rights reserved.

CSS Box Model

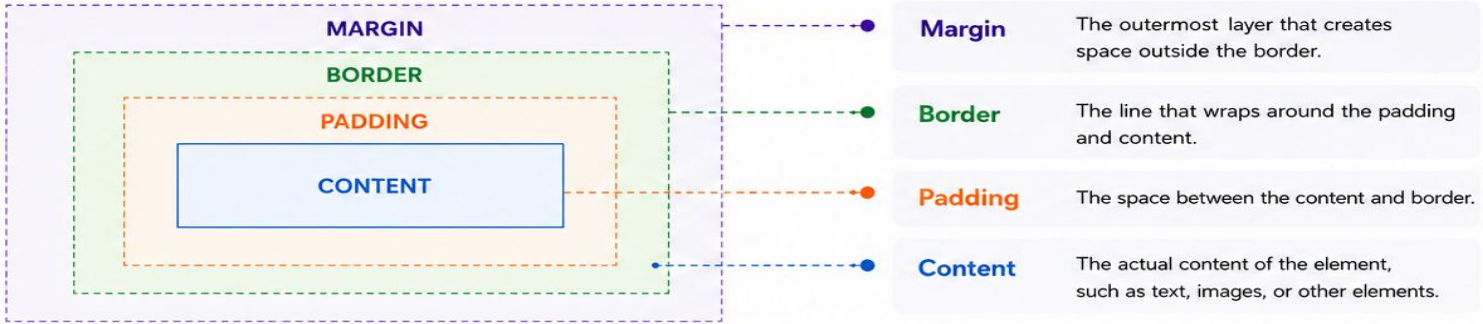
© Copyright Intellipaat. All rights reserved.

CSS Box Model

The **CSS Box Model** is the model the browser uses to represent and calculate the **size and space occupied by every HTML element** on a webpage.

The key idea is that an HTML element is treated as a **rectangular box**, and that box is made up of four layers:

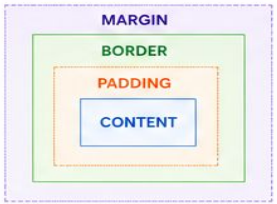
Content → Padding → Border → Margin



Margin	The outermost layer that creates space outside the border.
Border	The line that wraps around the padding and content.
Padding	The space between the content and border.
Content	The actual content of the element, such as text, images, or other elements.

Examples

1. Visual Representation



2. Box Model in Action



3. Dimensions Calculation

Total Width = ML + BL + PL + CW + PR + BR + MR

Total Height = MT + BT + PT + CH + PB + BB + MB

ML/MR = Margin Left/Right	MT/MB = Margin Top/Bottom
BL/BR = Border Left/Right	BT/BB = Border Top/Bottom
PL/PR = Padding Left/Right	PT/PB = Padding Top/Bottom
	CW/CH = Content Width/Height

4. CSS Example

```css
.box {
  width: 200px;
  padding: 20px;
  border: 5px solid #4CAF50;
  margin: 30px;
  background-color: #f0f8ff;
}
```

© Copyright Intellipaat. All rights reserved.

Content, Padding, Border and Margin

1 Content

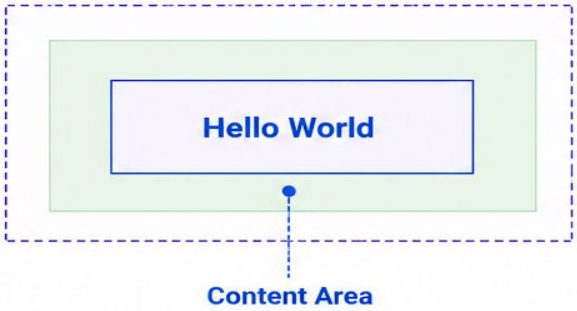
The **content area** is where the actual content of an element exists.

For example:

```
<p>Hello World</p>
```

The text `Hello World` occupies the content area.

CSS properties such as `width` and `height` can be used to control the dimensions of this area, depending on the element's `box-sizing`.



2 Padding

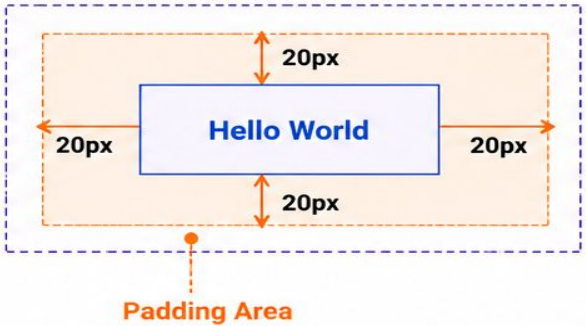
Padding is the space between the content and the border.

For example:

```
.box {
  padding: 20px;
}
```

This creates `20px` of space around the content **inside the element.**

Padding is useful when content feels too close to the element's boundary.



© Copyright Intellipaat. All rights reserved.

Content, Padding, Border and Margin

3 Border

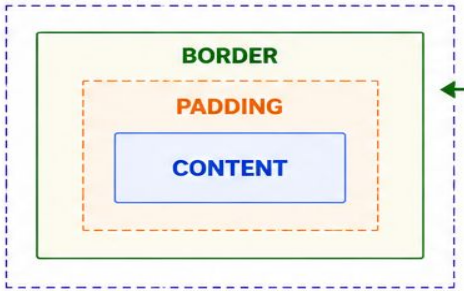
The **border** surrounds the padding and content.

```
.box {
    border: 2px solid black;
}
```

A border has three main characteristics:

- **Width** — how thick it is
- **Style** — solid, dashed, dotted, etc.
- **Color** — the border's color

The border is part of the element's box and can contribute to its total size.



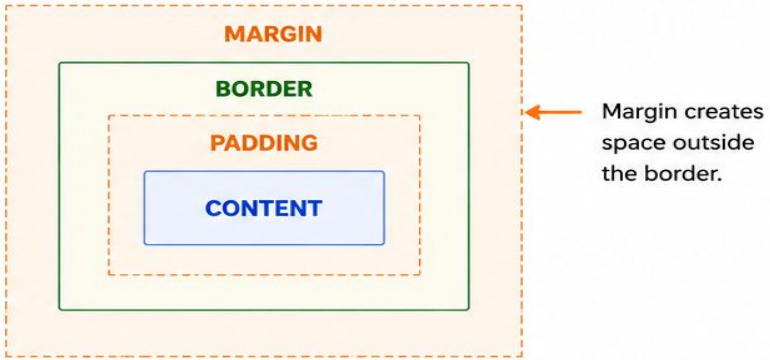
4 Margin

Margin is the space outside the border.

```
.box {
   margin: 20px;
}
```

It creates space between the element and the elements around it.

Unlike padding, margin does **not create** space **inside the element**.



© Copyright Intellipaat. All rights reserved.

Padding vs. Margin

This is one of the most important distinctions students need to understand:

Padding → Space **inside** the border

Margin → Space **outside** the border

For example:



How the Four Areas Work Together

Think of the box as layers:

1. **Content** — actual content
2. **Padding** — space around the content
3. **Border** — boundary around the box
4. **Margin** — space outside the box

This model explains why changing padding, borders, or margins can change how an element occupies space on the page.

© Copyright Intellipaat. All rights reserved.

Understanding Box Dimensions

Width of the Box

A box can be thought of as having:

Content width + Padding + Border

For example:

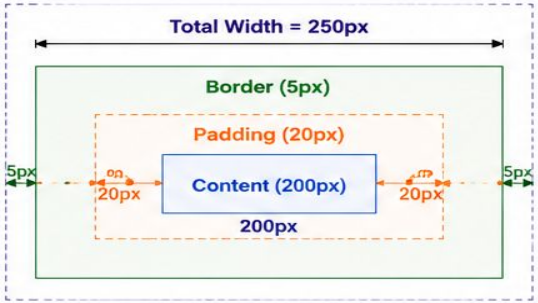
```
.box {
   width: 200px;
   padding: 20px;
   border: 5px solid black;
}
```

With the default `box-sizing: content-box`, the calculation is:

Content width:	200px
Left + right padding:	40px
Left + right border:	10px

So the total width becomes:

200 + 40 + 10 = 250px



Height of the Box

The same principle applies vertically:

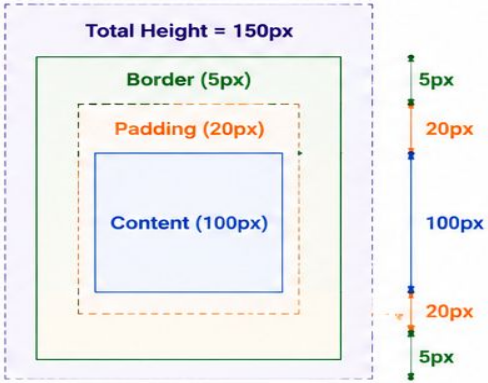
Content height + Top/bottom padding + Top/bottom border

For example:

```
.box {
   height: 100px;
   padding: 20px;
   border: 5px solid black;
}
```

The total height becomes:

100 + 40 + 10 = 150px



Margin

Margin is **not part of the element's box itself.** It creates space around the box.

So if the element above also has:

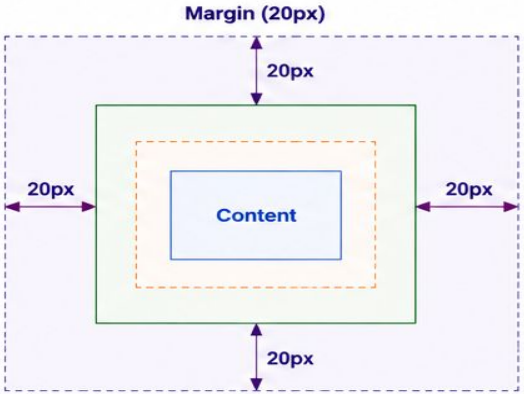
```
margin: 20px;
```

the margin adds space outside the element.

This distinction is important:

Content + Padding + Border → Box size

Margin → Space around the box



© Copyright Intellipaat. All rights reserved.

Hands On: 01

1

Start with the HTML

- Open the existing `index.html` project.
- Locate the existing course card.

2

Open the CSS

- Open `style.css`.
- Select the course card for styling.

3

Set the card size

- Give the course card a fixed width.
- Save and check the browser.

4

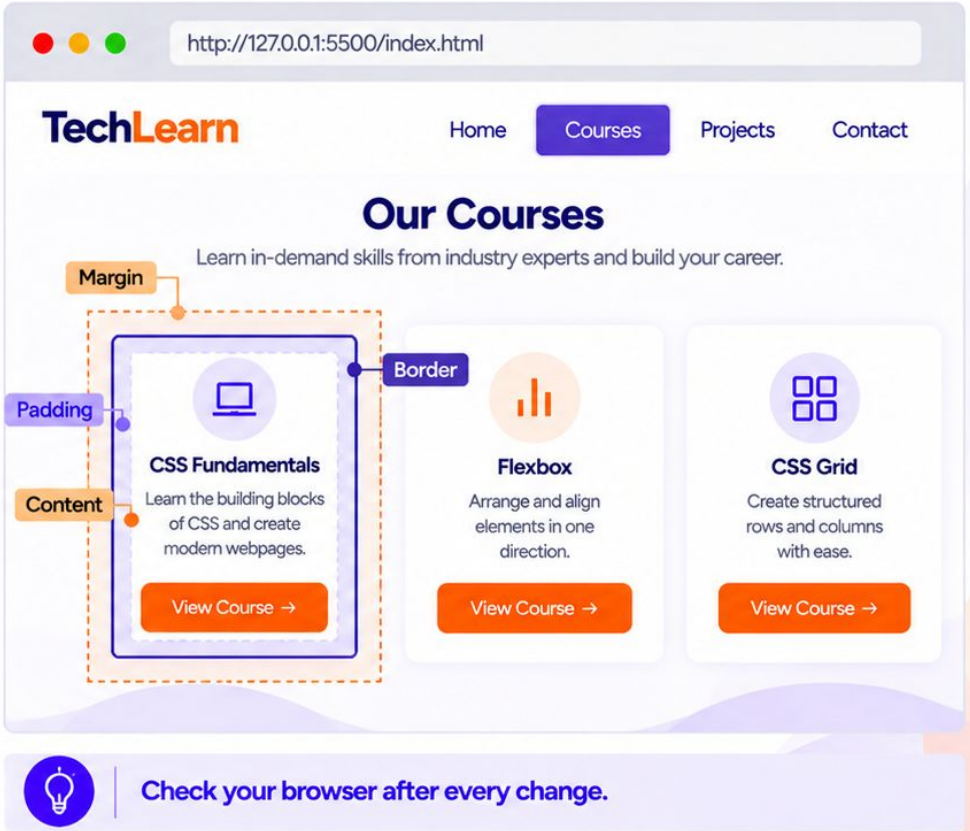
Build the box

- Add space inside the card content.
- Add a visible boundary around the card.
- Add space outside the boundary.

5

Observe the result

- Save and refresh the browser after each change.
- Identify Content, Padding, Border and Margin around the card.



Check your browser after every change.

© Copyright Intellipaat. All rights reserved.

Margin, Padding and Borders

© Copyright Intellipaat. All rights reserved.

Margin

Margin is the space **outside an element's border.** It controls the distance between an element and the elements around it.

If padding creates space **inside** the box, margin creates space **outside** the box.

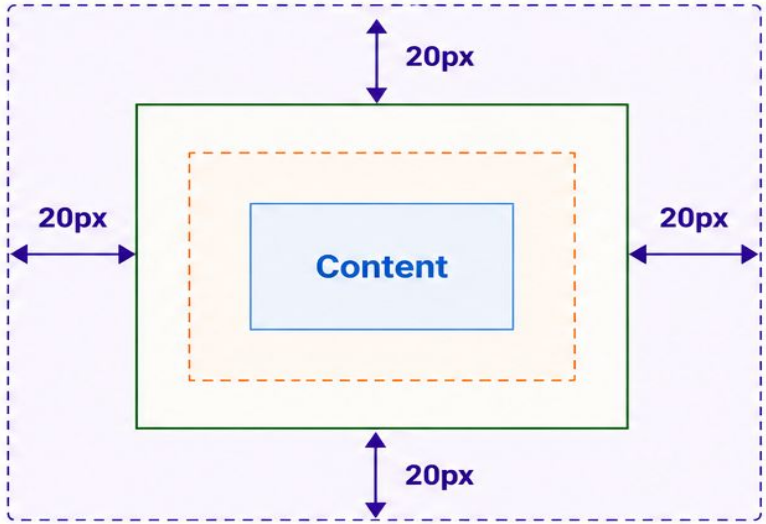
How Margin Works

For example:

```
.box {
  margin: 20px;
}
```

This creates **20px** of margin around the element:

- **Top**
- **Right**
- **Bottom**
- **Left**



So the element gets space between itself and surrounding elements.

Margin helps create space between the element and the surrounding elements.

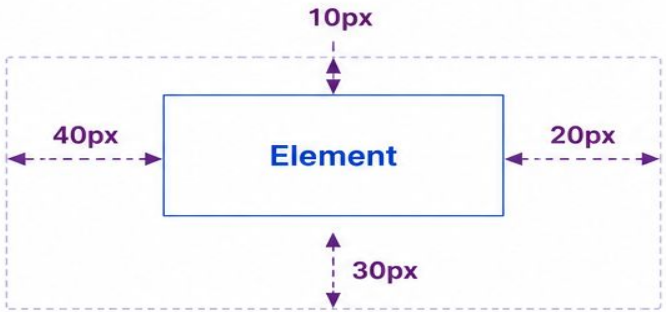
© Copyright Intellipaat. All rights reserved.

Margin

Individual Sides

Margin can be controlled separately for each side:

```
.box {
   margin-top: 10px;
   margin-right: 20px;
   margin-bottom: 30px;
   margin-left: 40px;
}
```



This is useful when different sides need different spacing.

auto Margin

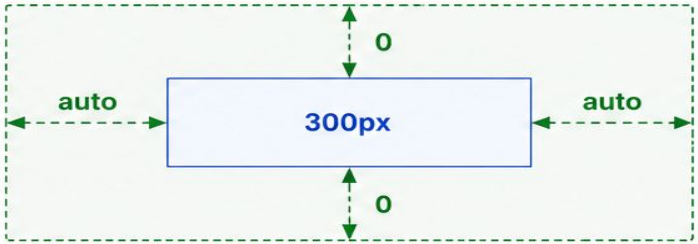
The `auto` value can be used to allow the browser to distribute available space.

A common example is horizontally centering a block element with a defined width:

```
.box {
   width: 300px;
   margin: 0 auto;
}
```

Here:

- `0` → top and bottom margin
- `auto` → left and right margins



The available horizontal space is distributed between the left and right margins, centering the box.

© Copyright Intellipaat. All rights reserved.

Margin

Margin Shorthand

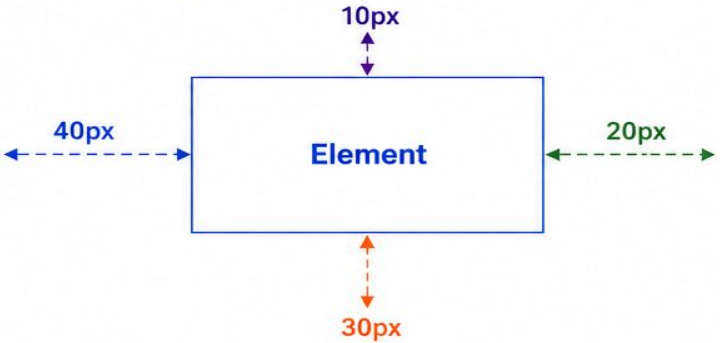
CSS also provides shorthand syntax

```
.box {
   margin: 10px 20px 30px 40px;
}
```

The values are interpreted in this order:

Top → Right → Bottom → Left

Students should understand this as the **clockwise order**.



You can also use fewer values:

```
margin: 20px 40px;
```

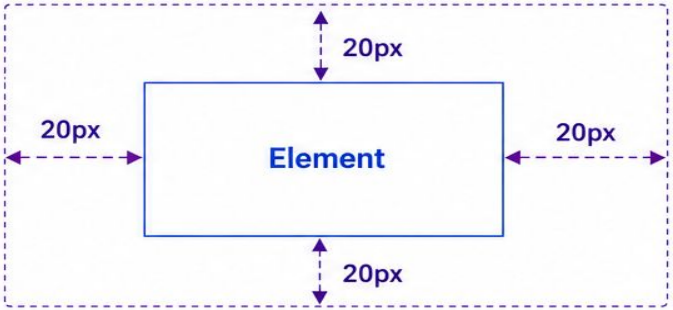
This means:

- Top & Bottom → 20px
- Left & Right → 40px

And:

```
margin: 20px;
```

means all four sides are 20px.



© Copyright Intellipaat. All rights reserved.

Padding

Padding is the space **inside an element, between its content and its border.**

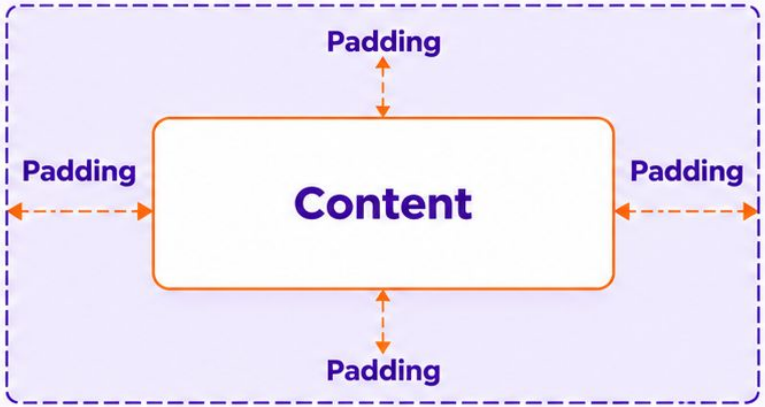
Unlike **margin**, which creates space outside the element, padding creates **internal space** around the content.

How Padding Works

For example:

```css
.box {
  padding: 20px;
}
```

This creates **20px** of space between the content and the border on all four sides.



Key Takeaway

Padding adds space inside the element, keeping the content away from the border.

© Copyright Intellipaat. All rights reserved.

Padding

Individual Sides

Padding can be controlled independently:

```css
.box {
   padding-top: 10px;
   padding-right: 20px;
   padding-bottom: 30px;
   padding-left: 40px;
}
```

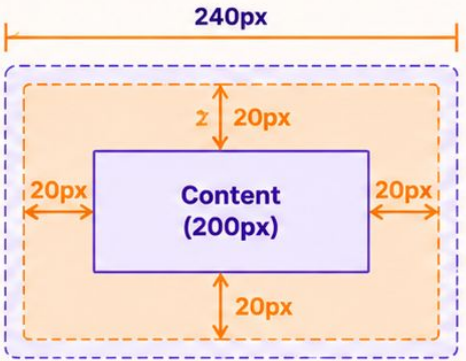
This allows different amounts of internal spacing on each side.

Padding Affects the Box Size

This is an important connection to the previous Box Model topic.

With the default `box-sizing: content-box`:

```css
.box {
   width: 200px;
   padding: 20px;
}
```



The `200px` refers to the **content width**, so the horizontal padding adds another `40px`.

Therefore, the content + padding width is:

200px + 20px + 20px = 240px

Borders, if present, would add further to the size.

Padding

Padding Shorthand

Like margin, padding supports shorthand:

```css
.box {
  padding: 10px 20px 30px 40px;
}
```

The order is:

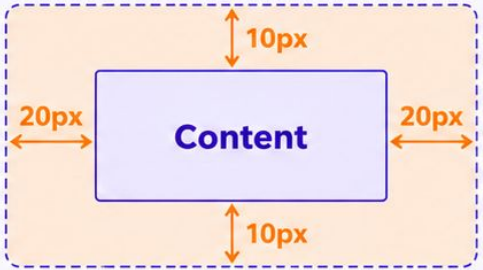
Top → Right → Bottom → Left

With fewer values

padding: 10px 20px;

means:

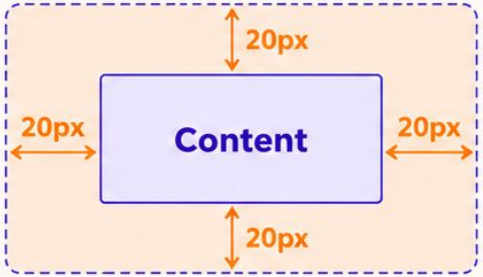
- Top & Bottom → **10px**
- Left & Right → **20px**



All sides equal

padding: 20px;

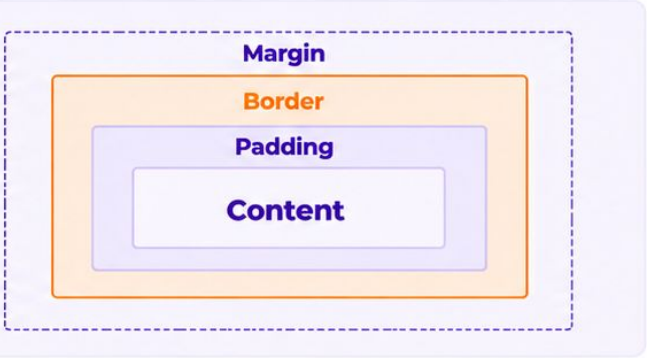
means **20px** on all four sides.



© Copyright Intellipaat. All rights reserved.

Borders

A **border** is the visible boundary around an element's content and padding. It sits **between the padding and the margin** in the CSS Box Model.



The Three Main Border Properties

A border is primarily defined using three characteristics.

1 Border Width

Controls the thickness of the border.

```css
.box {
   border-width: 2px;
}
```

2 Border Style

Controls how the border is drawn.

Common styles include:

- solid
- dashed
- dotted
- double

```css
.box {
   border-style: solid;
}
```

3 Border Color

Controls the border's color.

```css
.box {
   border-color: black;
}
```

These can be combined using the **border shorthand:**

```css
.box {
   border: 2px solid black;
}
```

This means:

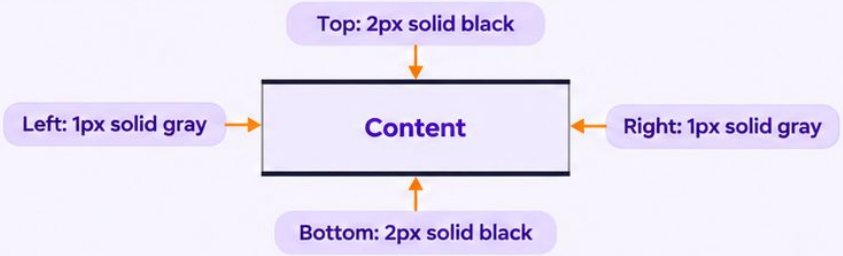
- 2px → width
- solid → style
- black → color

Borders

Borders on Individual Sides

A border can also be controlled separately for each side:

```css
.box {
   border-top: 2px solid black;
   border-right: 1px solid gray;
   border-bottom: 2px solid black;
   border-left: 1px solid gray;
}
```



This is useful when different sides need different borders.

Border and Box Dimensions

A border is part of the element's box.

With the default `box-sizing: content-box`, if you have:

```css
.box {
   width: 200px;
   border: 5px solid black;
}
```

The `200px` refers to the content width, and the left and right borders add another `10px` to the rendered width.

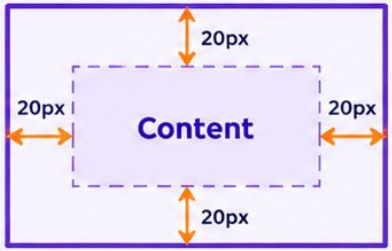


Margin vs. Padding

Padding

Padding is the space **inside** the element, between the content and the border.

```css
.box {
  padding: 20px;
}
```



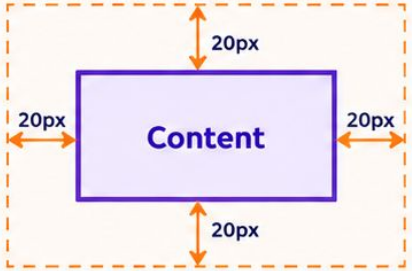
It pushes the content **away from the border.**

If an element has a background color, the background generally **extends through the padding area.**

Margin

Margin is the space **outside** the element's border.

```css
.box {
  margin: 20px;
}
```



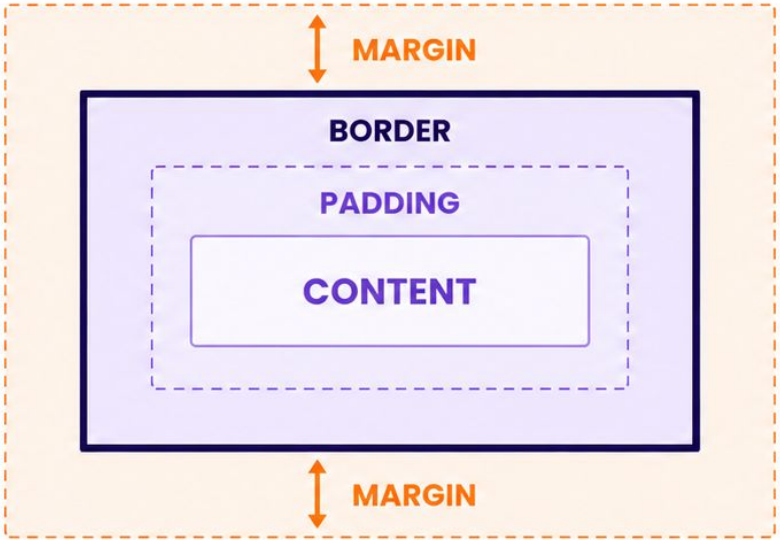
It creates distance **between the element and surrounding elements.**

The element's background **does not extend** into its margin.

Margin vs. Padding

Visual Difference

Think about the position of each:



Example

Suppose you have two boxes:

```HTML
<div class="box">Box 1</div>
<div class="box">Box 2</div>
```

If you use:

```CSS
.box {
   padding: 20px;
}
```

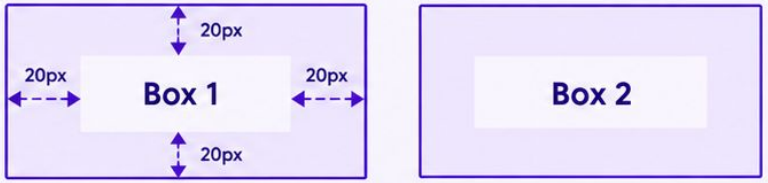
the text inside each box gets more space around it.

If you use:

```CSS
.box {
   margin: 20px;
}
```

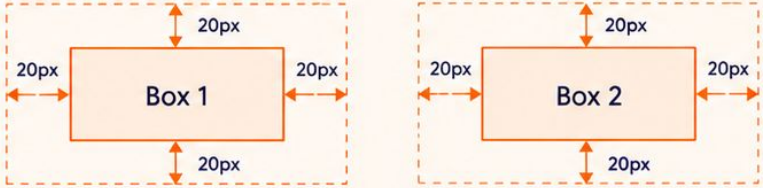
the two boxes get more space between them.

1 | Using Padding



The text inside each box gets **more space around it.**

2 | Using Margin



The two boxes get **more space between them.**

Hands On: 02

Use CSS spacing properties to **control how the course card sits inside the page.**

1 Continue with the course card

- Open the same `index.html` project.
- Locate the existing course card.

2 Adjust individual sides

- Open `style.css`.
- Experiment with top, right, bottom and left spacing.

3 Use shorthand

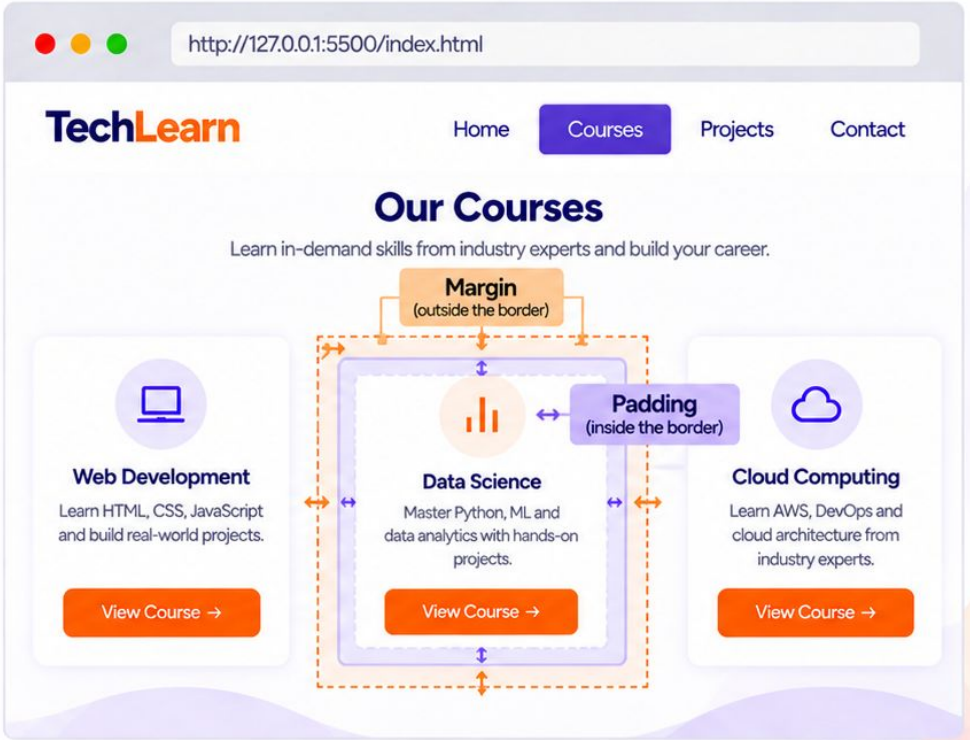
- Replace individual spacing rules with shorthand.
- Observe how multiple sides can be controlled together.

4 Center the card

- Use automatic horizontal spacing.
- Adjust the surrounding spacing until the card is positioned correctly.

5 Check the browser

- Save and refresh the page.
- Compare the space inside the card with the space outside it.



Adjust → Save → Refresh → Observe
Make each change and check how the spacing affects the card.

Width, Height and Box Sizing

© Copyright Intellipaat. All rights reserved.

Width and Height

The CSS **width** and **height** properties control the **size of an element.**

width → **controls how** wide an element is
height → **controls how** tall an **element is**

width

The **width** property controls the **horizontal size** of an element.

```css
.box {
   width: 300px;
}
```

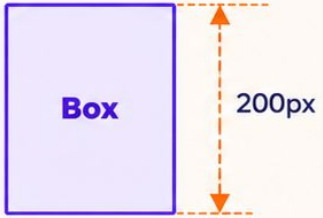


The exact part of the box that this value applies to depends on the element's **box-sizing** value, which students will learn shortly.

height

The **height** property controls the **vertical size** of an element.

```css
.box {
   height: 200px;
}
```



Again, whether this represents the content area or the overall box depends on **box-sizing**.

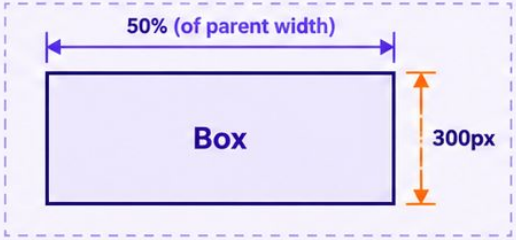
Width and Height

Width and Height Can Use Different Units

The dimensions don't have to be specified only in pixels.

For example:

```css
.box {
  width: 50%;
  height: 300px;
}
```



CSS units such as **px, %, em, rem, vh,** and **vw** can be used depending on the requirement.

width and height Do Not Automatically Include Everything

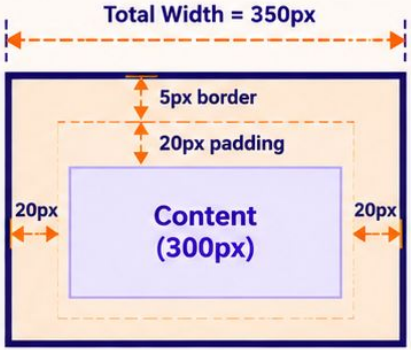
This is an important connection to the Box Model.

With the default:

`box-sizing: content-box,`

if you write:

```css
.box {
  width: 300px;
  padding: 20px;
  border: 5px solid black;
}
```



the 300px refers to the **content width.**

The actual horizontal space becomes:

300px content + 40px padding + 10px border = 350px

The same principle applies vertically to **height.**

Width and Height

min-width, max-width, min-height, max-height

These properties allow developers to place limits on an element's dimensions.

For example:

```css
.box {
  width: 100%;
  max-width: 600px;
}
```



This allows the box to grow with available space but prevents it from becoming wider than **600px**.

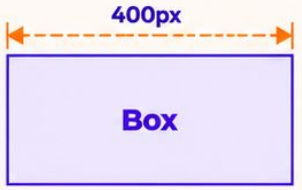
For this module, students only need to understand their basic purpose rather than going deeply into sizing algorithms.

Fixed vs. Flexible Dimensions

1 Fixed Dimension

A dimension such as:

```css
.box {
  width: 400px;
}
```

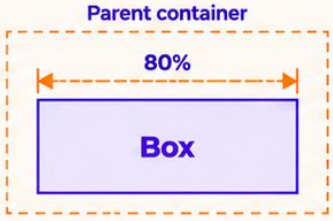


provides a **fixed CSS measurement.**

2 Flexible Dimension

While:

```css
.box {
  width: 80%;
}
```



allows the width to **change relative to its containing context.**

Understanding box-sizing

IntelliPaat

The box-sizing property controls **how the browser calculates the width and height of an element.**
This property is important because, as students learned earlier, an element consists of:

Content + Padding + Border

Two Main Values

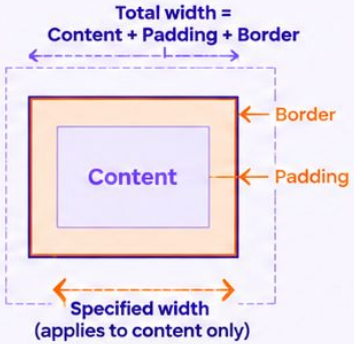
There are two values students need to understand.

1 content-box

```css
box-sizing: content-box;
```

The specified **width** and **height** apply to the **content area only**.

Padding and borders are added outside those dimensions.



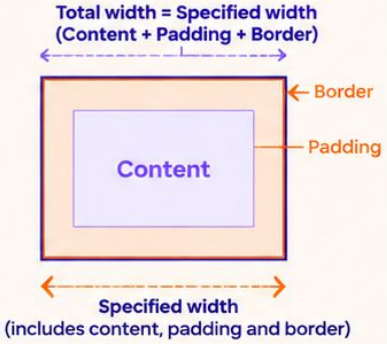
This means the actual size of the element becomes **larger** than the specified width and height.

2 border-box

```css
box-sizing: border-box;
```

The specified **width** and **height** include:

Content + Padding + Border



This means the element's total size will be **exactly** the specified width and height.

Understanding box-sizing

Why `box-sizing` Is Needed

Consider:

```css
.box {
   width: 300px;
   padding: 20px;
   border: 5px solid black;
}
```

You might expect the box to remain exactly **300px** wide.

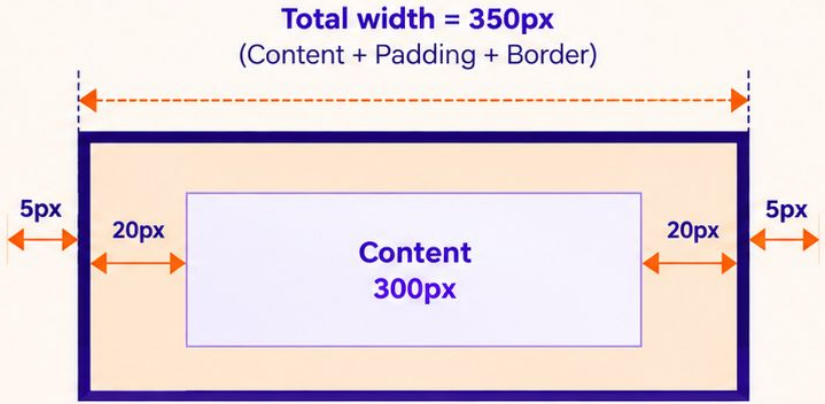
But with CSS's default sizing behavior, the **300px** applies only to the **content area**.

So the actual width becomes:

Content	300px
Padding (20px left + 20px right)	40px
Border (5px left + 5px right)	10px
Total	**350px**

`box-sizing` allows us to control this behavior.

Visual Representation



The actual width is **larger** than the specified width because padding and borders are added outside the **content area.**

Hands On: 03

Change the dimensions of the course card and see how **box sizing affects the final size.**

1 Select the course card

- Open the existing project.
- Locate the `.course-card` element.

2 Set dimensions

- Open `style.css`.
- Set the card's width and height.
- Refresh the browser and observe the size.

3 Add padding and border

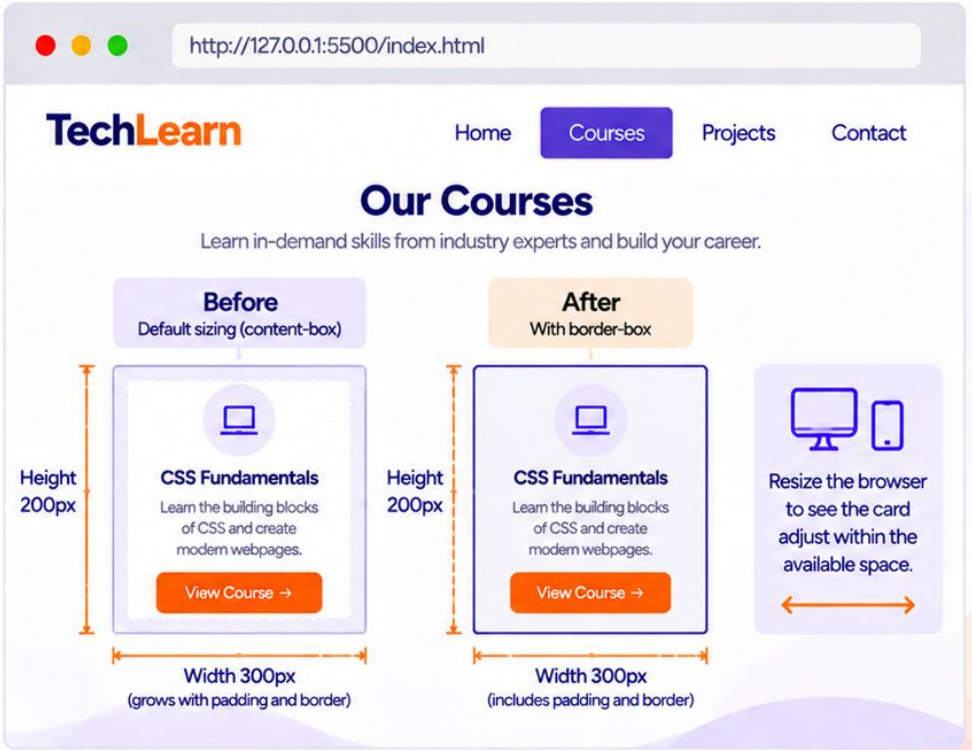
- Add internal spacing and a border.
- Observe how the overall card size changes.

4 Change box sizing

- Compare the default sizing behavior with `border-box`.
- Observe how the declared dimensions now include padding and border.

5 Make the card flexible

- Alllow the card to use available space.
- Limit its maximum width.
- Resize the browser and observe the result.



Resize the browser and observe how the card responds.

Display Properties

© Copyright Intellipaat. All rights reserved.

CSS Display Property

The `display` property controls **how an HTML element participates in the layout** and how it is represented in relation to surrounding elements.

Basic Idea

For example:

```css
div {
   display: inline;
}
```

This changes how the `<div>` participates in the page layout.

The `display` property therefore allows CSS to **change an element's default layout behavior.**

By changing the `display` value, you can control how the element is placed and how it interacts with surrounding elements.

Visual Representation

The value of `display` affects how the element is shown and how it flows with other elements.

1. display: inline

Elements stay in the same line

Item 1 Item 2 Item 3

Inline elements sit next to each other and only take up as much width as needed.

2. display: block

Elements start on a new line

Item 1

Item 2

Item 3

Block elements take up the full available width and start on a new line.

CSS Display Property

IntelliPaat

The Main Display Values

Students only need to focus on: **block**, **inline** & **inline-block**

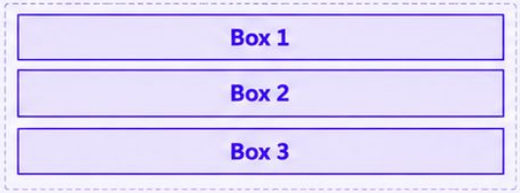
display: block

A block element takes up the full available width and starts on a new line.

CSS

```
div {
  display: block;
}
```

Output:



Each element appears on a new line and takes the full width available.

display: inline

An inline element does not start on a new line and only takes up as much width as its content.

CSS

```
span {
  display: inline;
}
```

Output:



Elements sit next to each other in the same line.

display: inline-block

An inline-block element sits in the same line like an inline element, but allows you to set width and height like a block element.

CSS

```
div {
  display: inline-block;
}
```

Output:



Elements sit next to each other and you can set width and height.

IntelliPaat

Block Elements

A block-level element normally:

- Starts on a **new line**
- Takes up the **available horizontal space** by default
- Can have `width` and `height`
- Can have `margin` and `padding` on all sides

For example:

HTML

```
<div>Box 1</div>
<div>Box 2</div>
```

The two `<div>` elements normally appear **one below the other**.

Visual Representation



Each `<div>` starts on a **new line** and takes the **full available width** (by default).

IntelliPaat

Inline Elements

An inline element normally:

- Stays within the **same line** as surrounding content
- Takes up only the **space required** by its content
- Does not normally accept **width** and **height** in the same way as block elements
- Can have **horizontal padding** and **margins**, but vertical layout behavior differs from block boxes

For example:

HTML

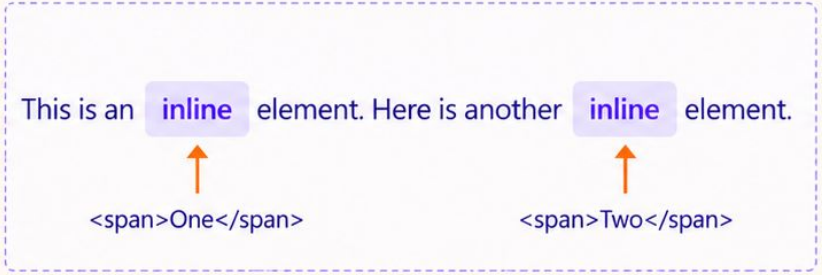
```
<span>One</span>
<span>Two</span>
```

They can appear **next to each other:**

One Two

Visual Representation

Inline elements flow within the same line and only take the space needed by their content.



Inline elements appear in the **same line** and take up only the **space required** by their content.

IntelliPaat

Inline-Block Elements

An **inline-block** element combines aspects of both **block** and **inline** behavior.

It:

- Can sit **next to other elements on the same line**
- Accepts `width` and `height`
- Supports **padding** and **borders** as a box
- Does not automatically take the full available width

For example:

CSS

```css
.box {
  display: inline-block;
  width: 150px;
  height: 100px;
}
```

Multiple boxes can therefore appear **beside each other** while still having controlled dimensions.

Visual Representation



These boxes appear **next to each other on the same line** because they use `display: inline-block`.

Inline-block gives you the flexibility to set **width, height, padding, and borders** while still keeping elements in the same line.

Comparing Block, Inline and Inline-Block

IntelliPaat

Feature	Block	Inline	Inline-Block
New line by default	Yes	No	No
Full available width by default	Usually	No	No
`width` / `height`	Yes	Generally no	Yes
Sits beside other elements	No, by default	Yes	Yes
Behaves as a controllable box	Yes	Limited	Yes

Use the right display value based on your layout needs. Each has different behavior, and understanding these differences helps you build better, more flexible web pages.

Hands On: 04

Change how the **navigation links** are displayed and see how it affects the **page layout**.

1 Locate the navigation

- Open the existing `index.html`.
- Locate the navigation links inside the `.nav` section.

2 Try block display

- Open `style.css`.
- Change the navigation links to block elements.
- Refresh and observe the vertical layout.

3 Try inline display

- Change the links to inline elements.
- Refresh and observe how they appear on the same line.

4 Try inline-block

- Change the links to `inline-block` elements.
- Add controlled spacing and dimensions.
- Observe how the links stay on the same line with a defined box size.

5 Compare the results

- Compare block, inline and inline-block behavior.
- Keep the version that gives the navigation the desired layout.



Change → Refresh → Compare

Try each display type and observe how the navigation layout changes.

CSS Positioning

© Copyright Intellipaat. All rights reserved.

CSS Positioning

CSS positioning controls how an element is positioned in relation to its **normal position, containing element, or viewport.**

The `position` property determines the positioning behavior, while properties such as `top`, `right`, `bottom`, and `left` can then be used to adjust the element's position.

The Main Positioning Values

For this module, students need to understand four positioning types:

1. relative

Positions the element relative to its normal position.



Moves from its original position using `top`, `right`, `bottom`, or `left`.

2. absolute

Positions the element relative to its nearest positioned ancestor (or the viewport if none).



Removed from the normal flow and positioned using `top`, `right`, `bottom`, or `left`.

3. fixed

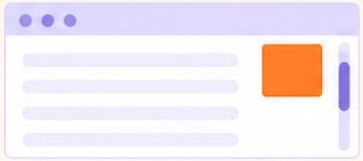
Positions the element relative to the viewport.



Stays in the same position even when the page is scrolled.

4. sticky

Toggles between relative and fixed positioning based on the scroll position.



Behaves like relative until a scroll threshold is reached, then sticks like fixed.

These values behave differently and are useful for different layout situations.

CSS Positioning

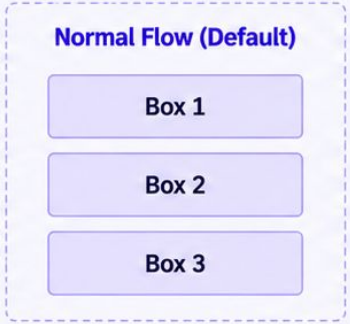
Normal Flow vs. Positioned Elements

By default, elements are placed according to the **normal flow** of the document.

For example:

```html
<div>Box 1</div>
<div>Box 2</div>
<div>Box 3</div>
```

The browser determines their positions based on the normal layout rules.



When you use:

```css
.box {
  position: relative;
}
```

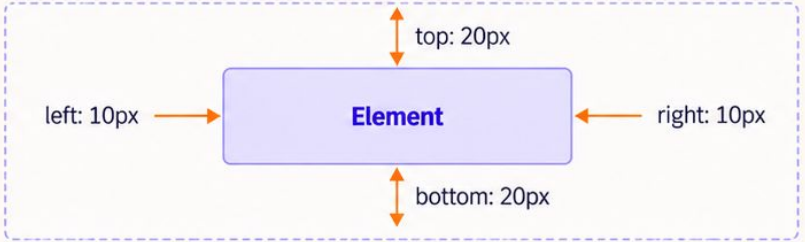
or another positioning value, you **change how that element participates in positioning.**

Positioning Properties

Once an element has a positioning mode, properties such as `top`, `right`, `bottom`, and `left` can control its position:

```css
top: 20px;
right: 10px;
bottom: 20px;
left: 10px;
```

These specify distances from the relevant reference edge.



These properties help you **fine-tune the position** of an element based on the chosen positioning value.

Relative Positioning

`position: relative` allows an element to be positioned relative to its **normal position in the document flow.**

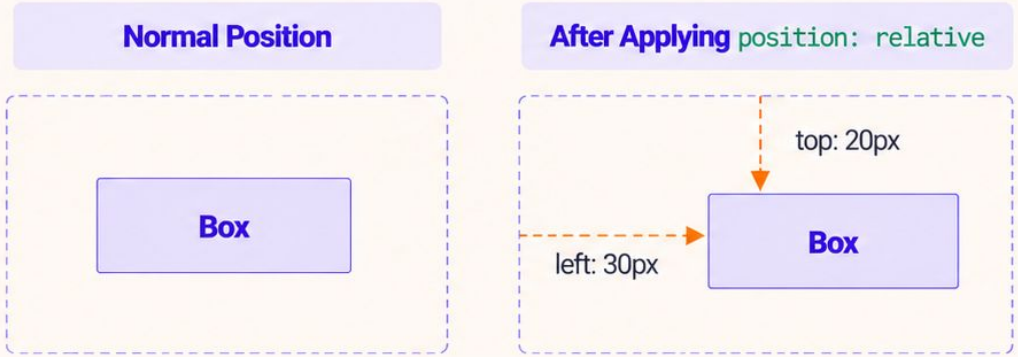
How It Works

For example:

```css
.box {
  position: relative;
  top: 20px;
  left: 30px;
}
```

Here, `top` and `left` move the element from where it would normally have been placed.

Visual Representation



Original position in the document flow.

Element moves 20px down and 30px right from its normal position.

The element **still remains in the document flow;** it is just visually shifted from its original position.

Relative Positioning

For example, if the element normally occupies a particular space and you move it using `top` or `left`,
its original space is still preserved.

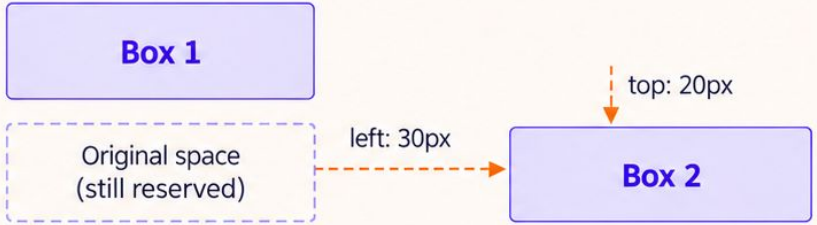
Normal Position



HTML

```
<div>Box 1</div>
<div>Box 2</div>
```

After Moving Box 2



CSS

```
.box2 {
  position: relative;
  top: 20px;
  left: 30px;
}
```

The space where Box 2 originally existed is **still reserved.**

Absolute Positioning

`position: absolute` removes an element from the **normal document flow** and positions it relative to its **containing block**.

This makes absolute positioning very different from relative positioning.

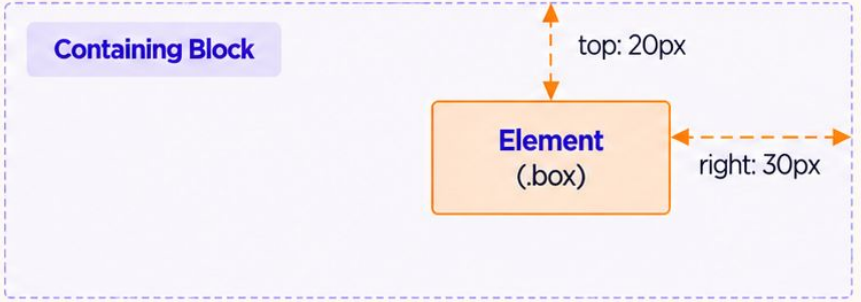
How It Works

For example:

```css
.box {
  position: absolute;
  top: 20px;
  right: 30px;
}
```

The browser positions the element according to its **containing block** and the specified offsets.

Visual Representation



The element is removed from the **normal document flow** and positioned relative to its **containing block**.

Use absolute positioning when you need **precise control** over an element's position within a container.

CSS Specificity

IntelliPaat

The most important concept is:
An absolutely positioned element is removed from the **normal document flow.**

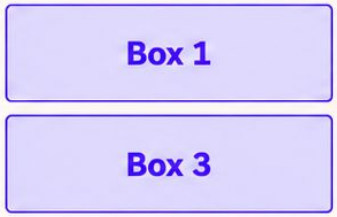
This means other elements behave as though the absolutely positioned element is **not occupying its original space.**

Before



All boxes are in the normal document flow.

After Box 2 → absolute



Box 3 moves into the space that Box 2 previously occupied.



Box 2 is positioned separately (absolute).

Absolute positioning is useful when you need to **place an element independently** without affecting the layout of other elements.

Fixed and Sticky Positioning

This slide covers the two positioning modes that are closely related to **scrolling and the viewport:** `fixed` and `sticky`.

The key difference is **how the element behaves while the page is scrolled.**

Fixed Positioning

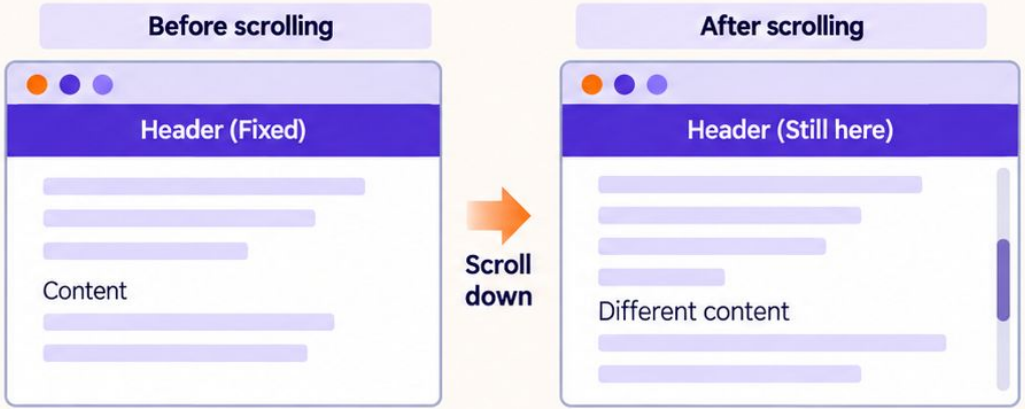
`position: fixed` positions an element relative to the **viewport.**

```css
.header {
  position: fixed;
  top: 0;
  left: 0;
}
```

The element is **removed from the normal document flow** and stays attached to the specified position in the viewport.

When the user scrolls the page, the element **stays in the same place on the screen.**

Conceptually:



Fixed positioning is useful for elements like **navigation bars, headers, or action buttons** that need to remain visible even while scrolling.

Fixed and Sticky Positioning

Sticky Positioning

`position: sticky` behaves like a **normally positioned element** until a specified scroll position is reached.

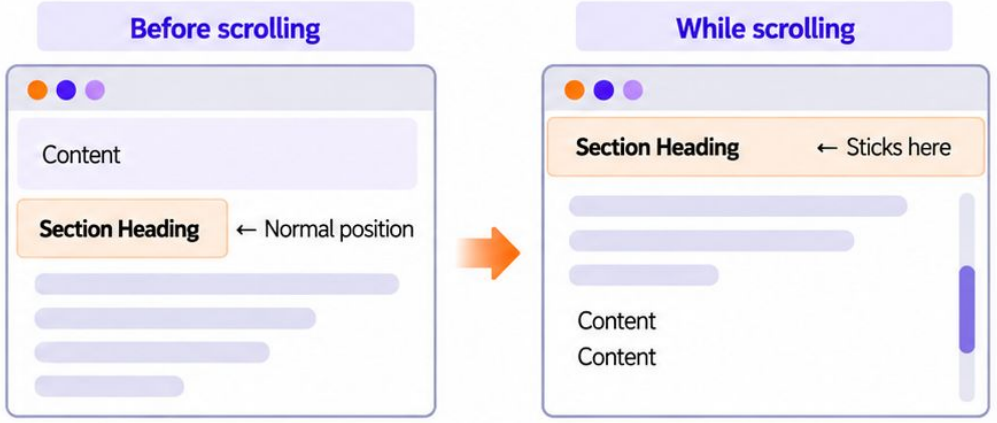
For example:

```css
.heading {
  position: sticky;
  top: 0;
}
```

Initially, the element behaves as part of the **normal flow**.

As the user scrolls and the element reaches `top: 0`, **it sticks to that position** within its scrolling context.

Conceptually:



Heading is in its normal position as part of the document flow.

Once the scroll position is reached, the heading **sticks to the top**.

Sticky positioning is useful for **section headings, navigation elements**, or any content that should **remain visible** within a scrolling container.

Fixed vs. Sticky

IntelliPaat

Aspect	Fixed 	Sticky 
Reference	Viewport	Scrolling/containing context
Normal flow	Removed	Initially remains in flow
Scroll behavior	Always stays fixed	Sticks after reaching its offset
Typical use	Persistent UI	Section-based sticky elements

Hands On: 05

Add a badge to the course card and position it precisely using **relative and absolute positioning.**

1 Start with the course card

- Open the existing project.
- Locate the course card.

2 Add a badge

- Add a small "NEW" label inside the course card.
- Place it near the card content.

3 Set the positioning reference

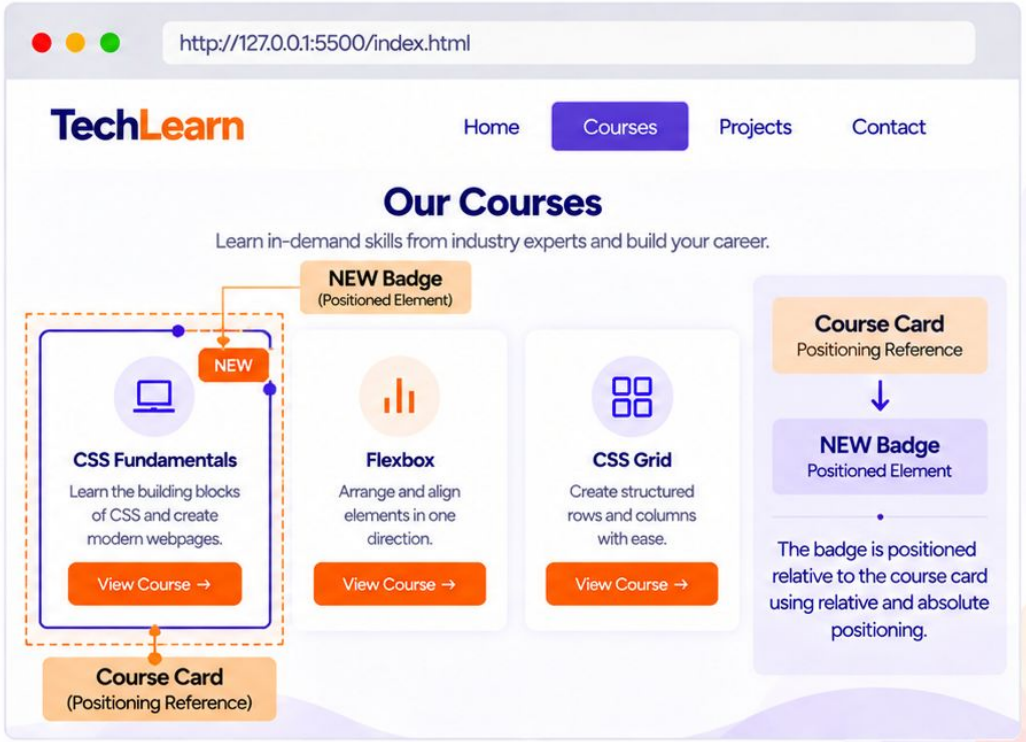
- Open `style.css`.
- Make the course card the reference for positioned content.

4 Position the badge

- Position the badge independently from the normal document flow.
- Move it toward the top-right corner.
- Adjust its offsets until it is correctly placed.

5 Check the browser

- Save and refresh the page.
- Confirm that the badge remains positioned relative to the course card.



Move the badge → Refresh → Observe its position.
Adjust the position and see how it stays within the course card.

Overflow and Z-Index

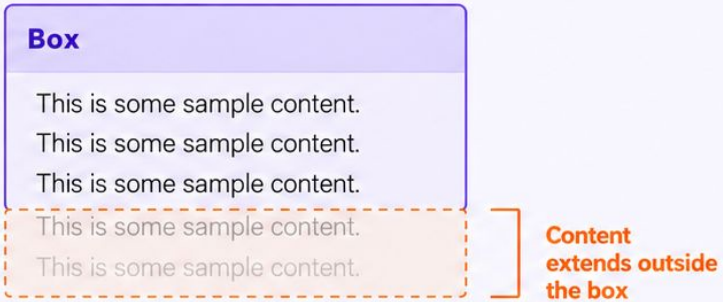
© Copyright Intellipaat. All rights reserved.

CSS Overflow

Overflow refers to what happens when an element's content is **larger than the space available inside its box.**

For example, if a box has a fixed height but contains more content than can fit, the content may extend outside the box.

CSS's `overflow` property controls how this extra content is handled.



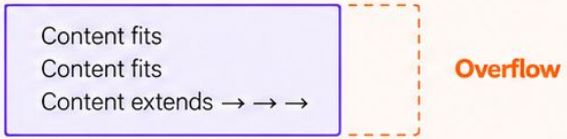
Why Overflow Happens

Consider:

```css
.box {
  width: 200px;
  height: 100px;
}
```

If the content inside the box requires more than `200px` of width or `100px` of height, it can overflow the element's boundaries.

Conceptually:



Use the CSS `overflow` property (e.g., **visible, hidden, scroll, auto**) to control how extra content is displayed.

CSS Overflow

The Main `overflow` Values

visible

This is the default behavior.

Content that doesn't fit can **extend outside the element's box.**

```css
.box {
  overflow: visible;
}
```

Content extends outside

This is a long content.
This is a long content.
This is a long content.
This is a long content.

hidden

Extra content is **clipped** and isn't visible outside the box.

```css
.box {
  overflow: hidden;
}
```

Content is clipped

This is a long content.
This is a long content.
This is a long content.

This is useful when content should not visually extend beyond a defined boundary.

scroll

The browser provides **scrollbars**, allowing the user to access content that doesn't fit.

```css
.box {
  overflow: scroll;
}
```

Scrollbars are shown

This is a long content.
This is a long content.
This is a long content.
This is a long content.

The exact scrollbar behavior can depend on the browser and platform.

auto

The browser determines whether scrolling is necessary.

```css
.box {
  overflow: auto;
}
```

Scrollbar appears only if needed

This is a long content.
This is a long content.
This is a long content.

If the content fits, scrolling isn't needed. If it overflows, the browser can provide the necessary scrolling mechanism.

Use the CSS `overflow` property (e.g., **visible, hidden, scroll, auto**) to control how extra content is displayed.

CSS Overflow

Horizontal and Vertical Overflow

Controlling Overflow Independently

Overflow can also be controlled independently:

```css
.box {
  overflow-x: auto;
  overflow-y: hidden;
}
```

`overflow-x` → controls horizontal overflow.

`overflow-y` → controls vertical overflow.

Why Overflow Matters

Overflow becomes important when working with:

- Fixed-size containers
- Long text or content
- Images
- Scrollable areas
- Components where content must stay within boundaries

It is also relevant to positioning and layout because overflow settings can affect **scrolling behavior and positioning contexts.**

Use `overflow-x` and `overflow-y` when you need **different behaviors** for horizontal and vertical overflow.

Z-Index and Stacking Order

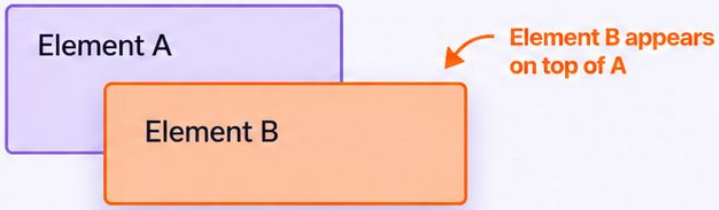
IntelliPaat

`z-index` controls the **stacking order of overlapping elements**, in other words, which element **appears in front** when elements occupy the same area.

Why Stacking Order Matters

Normally, elements are arranged according to the browser's layout and painting rules. But when elements overlap, one may appear **on top of another**.

For example:



If A and B overlap, stacking order determines which one is **visible on top.**

Using z-index

You can assign a `z-index` value:

CSS

```css
.box-a {
  position: relative;
  z-index: 1;
}

.box-b {
  position: relative;
  z-index: 2;
}
```

Here, `.box-b` has the higher `z-index`, so it is placed **above** `.box-a` when they overlap within the relevant stacking context.

`z-index` works only on **positioned elements** (e.g., relative, absolute, fixed, sticky) within the same stacking context.

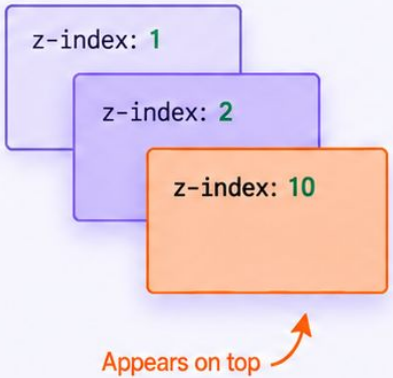
Z-Index and Stacking Order

A Simple Rule

Higher `z-index` → higher stacking position

For example:

`z-index: 1`	→	behind
`z-index: 2`	→	in front
`z-index: 10`	→	further in front



z-index Is Not a Positioning Property

A common misunderstanding is that `z-index` controls where an element is positioned.

It does not control:

- Top
- Bottom
- Left
- Right
- Width
- Height

It controls which overlapping element is **painted in front**.

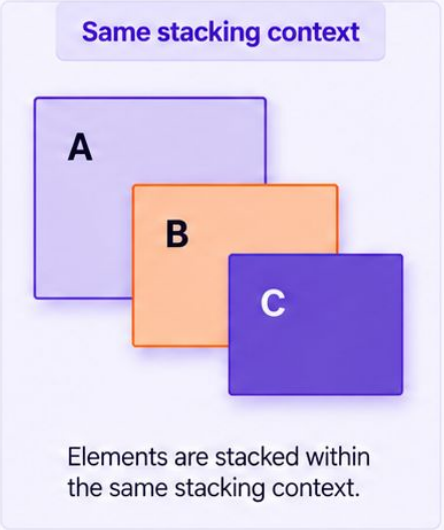
Positioning and stacking are separate concepts.

Use `z-index` only to manage the **stacking order** of overlapping elements, not to set their position.

Z-Index and Stacking Order

Stacking Context

- **z-index** works within **stacking contexts.**
- A stacking context is a group of elements whose stacking order is managed together.
- A child with a high z-index cannot appear above an element outside its stacking context.
- For this module, focus on understanding **overlapping elements and stacking order.**



Elements are stacked within the same stacking context.

Common Uses

z-index is commonly useful for:

- Dropdowns
- Modals
- Tooltips
- Overlays
- Navigation elements
- Elements that need to appear above others

Key Takeaway: **z-index** controls the **stacking order within a stacking context,** not the element's position.

Hands On: 06

Create a constrained content area, handle overflowing content, and control the **stacking order** of elements.

1 Create a constrained content area

- Open the existing project.
- Select the Course Details section.
- Give the section limited dimensions.

2 Create overflow

- Add enough content to exceed the available space.
- Refresh the browser.
- Observe what happens when content no longer fits.

3 Control the overflow

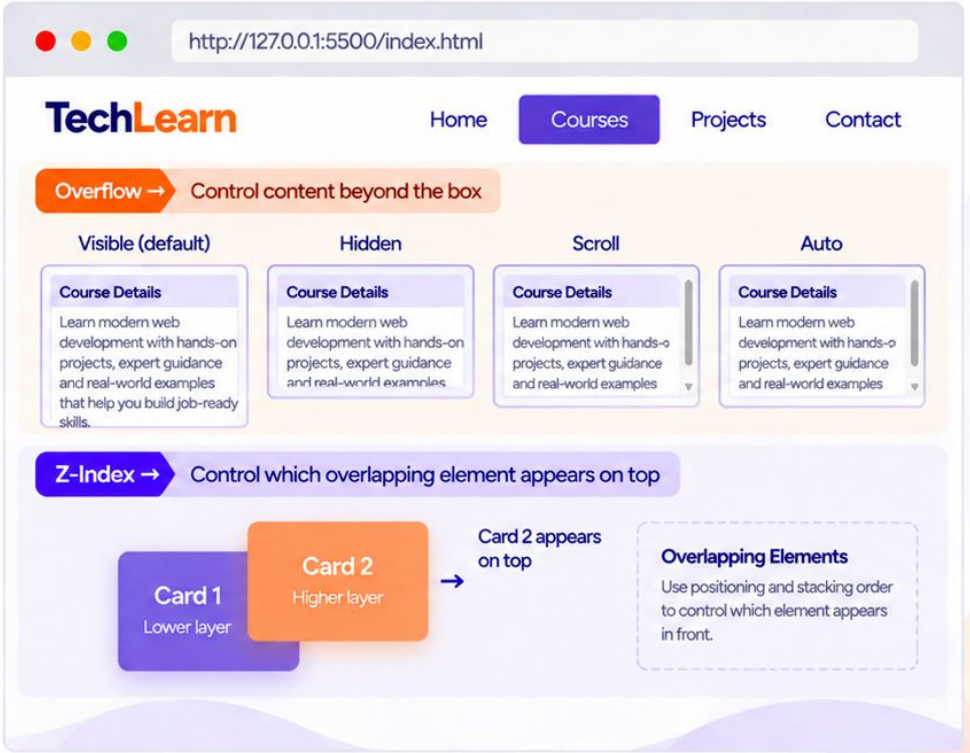
- Try different overflow behaviors.
- Compare visible, hidden, scrollable and automatic behavior.
- See how each behavior handles the extra content.

4 Create overlapping elements

- Add two positioned elements.
- Move them so that they overlap.
- Observe which element appears in front.

5 Control the stacking order

- Change the stacking levels of the elements.
- Refresh and confirm which element appears on top.



Thanks!



080-4524-9465



support@intellipaat.com

© Copyright Intellipaat. All rights reserved.